



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 微处理器原理与应用

实验名称: 实验三

子程序汇编实验学习和提高

专 业 班 级 通信工程 二班

学 生 学 号 202400120045

学 生 姓 名 刘哲旭

实 验 时 间 2025 年 3 月 25 日

实验报告

【实验目的】

1. 理解汇编语言中的 ASSUME 伪指令和标准的汇编程序
2. 复习 Debug -P 和-G
3. 将键盘上输入的十六进制数转换成十进制数，并在屏幕上显示。（编写程序，详细注释并画程序流程图）
4. 较为复杂的汇编实例学习：判断该年是否为闰年(通过注释重点学习并理解程序，画出程序的流程图，在理解的基础上，如果觉得程序的写法需要修改也可以自行修改)
5. 汇编实例学习和改进：两位数加法

【实验要求】

- 1.
- 2.

【实验具体内容】

【第一个实验】Debug -P 和-G

在调试过程中，P 指令与 T 指令的功能有所不同。当程序只有单行语句时，两者作用相同，都会执行当前一行代码。但当面对包含子程序、循环或中断的代码段时，P 指令会直接一次性执行完整个程序段，而 T 指令则会逐条指令进入内部，单步执行。

T 指令的作用是从 CS:IP 指向的位置开始，执行一条或多条指令，并在每条指令执行完成后，显示所有寄存器的当前状态。

P 指令主要用于执行循环、重复字符串操作、软中断以及子程序调用等情况，它不会进入子程序内部，而是直接将整个调用视为一步完成。

(1) 实验源代码（粘贴源代码）：

```
MOV AX,2; 将 2 赋值给寄存器 AX。
ADD AX,AX; 将寄存器 AX 内的值加给 AX。
ADD AX,AX; 将寄存器 AX 内的值加给 AX。
ADD AX,AX; 将寄存器 AX 内的值加给 AX。
```

(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

①在 DOSBox 中，使用-r 命令设置 CS=0C52，IP=0100，再用-r 指令查看修改后各寄存器中的值，如图 1-1 所示。

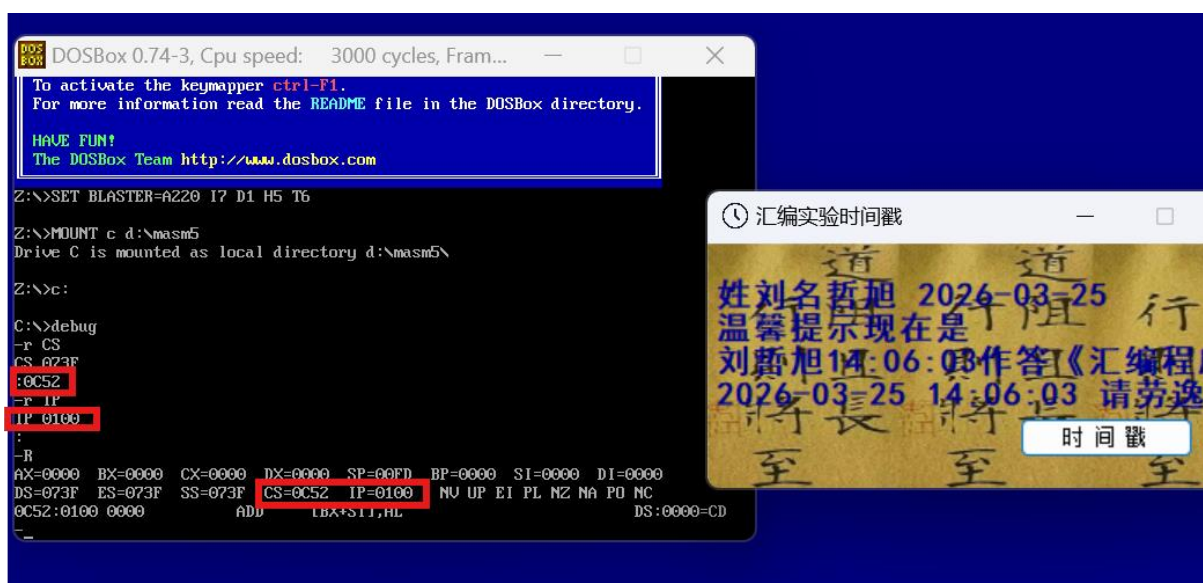


图 1-1 修改 CS 和 IP 中的值

②使用-a 指令向代码段写入以下指令，再使用-u 指令查看刚刚写入的代码，如图 1-2 所示。

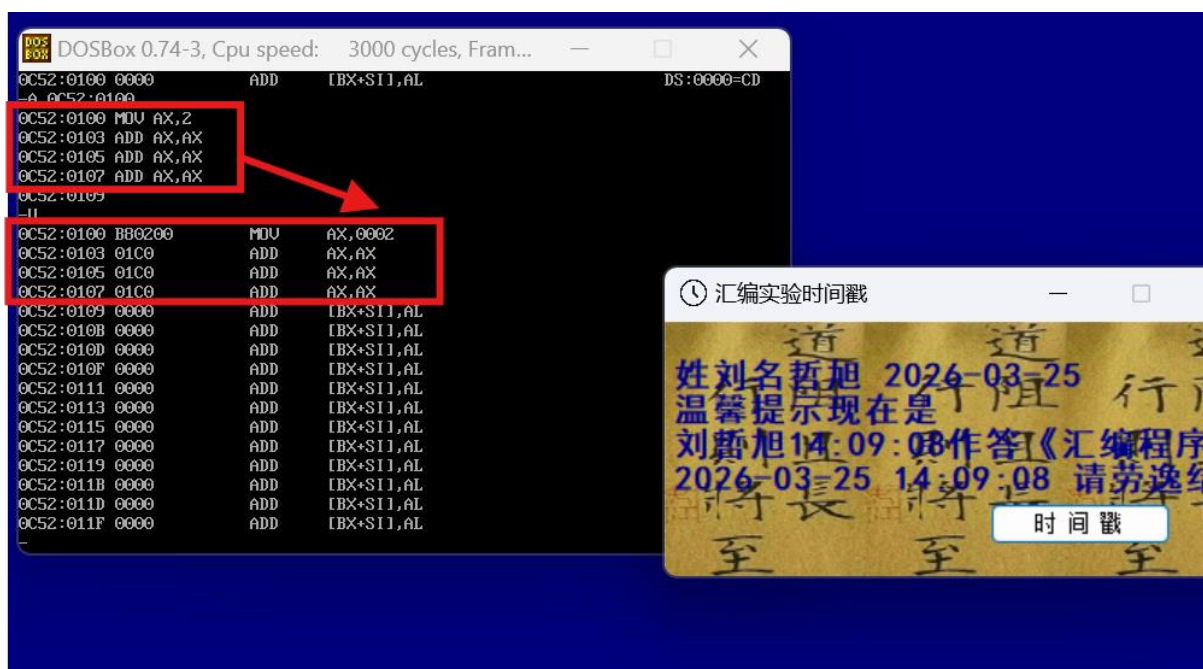


图 1-2 -a 写入指令后查看输入的代码

③输入-g 指令输入-g 0C52:0109，程序直接运行至指定断点处，IP 变为 0109H，输入-g 010F，程序直接运行至指定断点处,IP 变为 010FH,如图 1-3 所示。



图 1-3 直接运行至断点

【第二个实验】将十六进制数转换成十进制（时间戳）

(1) 实验流程图：

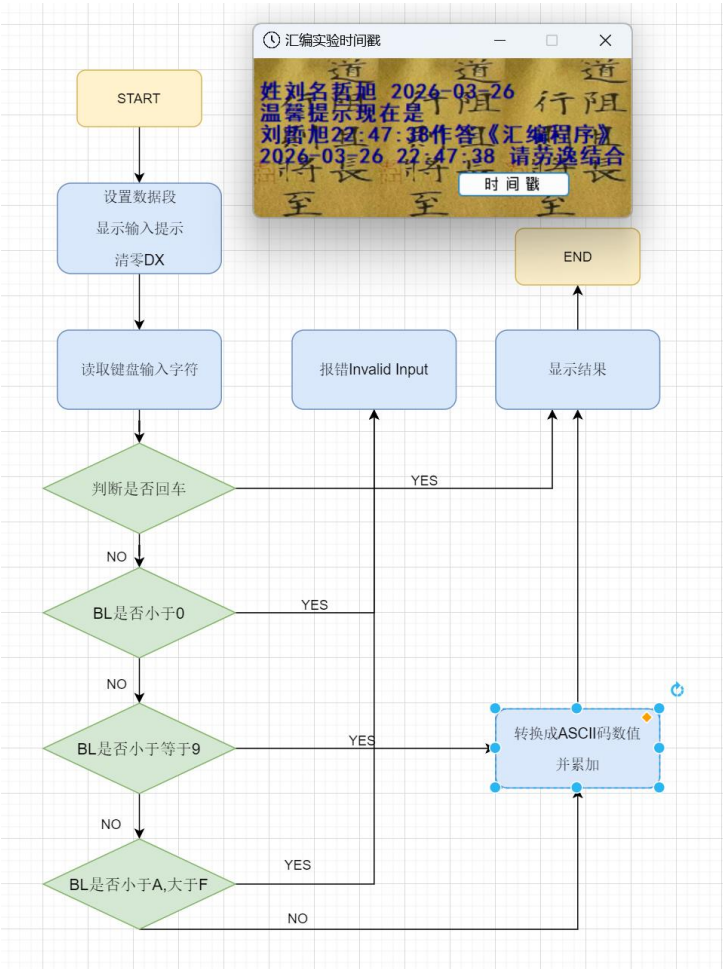


图 2-1：流程图

(2) 实验源代码（粘贴源代码）：

DATA_SEG SEGMENT ; 数据段开始

MSG_INPUT DB 13,10,'Enter a hexadecimal number (up to 4
digits):',13,10,'\$' ; 定义输入提示字符串

MSG_ERROR DB 13,10,'Invalid input! Please try again.',13,10,'\$' ; 定义错误提示字符串

DATA_SEG ENDS ; 数据段结束

STACK_SEG SEGMENT STACK ; 栈段开始

DW 64 DUP (0) ; 定义64个字的栈空间

STACK_SEG ENDS ; 栈段结束

CODE_SEG SEGMENT ; 代码段开始

ASSUME CS:CODE_SEG,DS:DATA_SEG,SS:STACK_SEG ; 指定段寄存器关联

MAIN: ; 主程序入口标签

MOV AX, DATA_SEG ; 将数据段地址加载到AX寄存器

MOV DS, AX ; 将数据段地址存入DS寄存器

LEA DX, MSG_INPUT ; 将输入提示字符串的偏移地址加载到DX

MOV AH, 09H ; 设置AH为09H, 准备显示字符串

INT 21H ; 调用DOS中断21H显示输入提示

XOR DX, DX ; 清零DX寄存器, 用于累积十六进制数值

READ_INPUT: ; 读取输入循环标签

MOV AH, 01H ; 设置AH为01H, 准备从键盘读取单个字符

INT 21H ; 调用DOS中断21H读取键盘输入到AL

MOV BL, AL ; 将输入字符从AL移动到BL

XOR BH, BH ; 清零BH寄存器, 保证BX高字节为0 (非常关键!)

CMP BL, 0DH ; 比较BL是否为回车键 (0DH)

JE DISPLAY_RESULT ; 如果是回车, 跳转到显示结果部分

CMP BL, '0' ; 比较BL是否小于字符'0'

JB INVALID_INPUT ; 如果小于'0', 跳转到无效输入处理

CMP BL, '9' ; 比较BL是否小于等于字符'9'

JBE CONVERT_DIGIT ; 如果是0-9数字, 跳转到数字转换

AND BL, 0DFH ; 将字母a-f转换为大写A-F

CMP BL, 'A' ; 比较BL是否小于字符'A'

JB INVALID_INPUT ; 如果小于'A', 跳转到无效输入

CMP BL, 'F' ; 比较BL是否大于字符'F'

JA INVALID_INPUT ; 如果大于'F', 跳转到无效输入

SUB BL, 'A'-10 ; 将A-F字母转换为数值10-15

JMP ADD_TO_DX ; 跳转到累加到结果部分

CONVERT_DIGIT: ; 数字转换标签

SUB BL, '0' ; 将ASCII数字字符'0'-'9'转换为实际数值0-9

ADD_TO_DX: ; 累加数值标签

MOV CL, 4 ; 设置左移位数为4

SHL DX, CL ; 将DX左移4位 (相当于当前值乘以16)

ADD DX, BX ; 将新转换的数字加到DX中

JMP READ_INPUT ; 跳转回继续读取下一个输入字符

INVALID_INPUT: ; 无效输入处理标签

LEA DX, MSG_ERROR ; 将错误提示字符串的偏移地址加载到DX

MOV AH, 09H ; 设置AH为09H, 准备显示字符串

INT 21H ; 调用DOS中断21H显示错误提示

JMP READ_INPUT ; 跳转回继续等待输入 (符合“try again”)

DISPLAY_RESULT: ; 显示十进制结果标签

MOV AX, DX ; AX = 要显示的十进制数值

MOV BX, 10 ; 设置除数为10

XOR CX, CX ; CX 计数压栈的位数

OR AX, AX ; 特殊处理0: 检查AX是否为0

JNZ CONVERT_LOOP ; 如果不为0, 跳转到转换循环

MOV DL, '0' ; 如果结果为0, 直接准备显示字符'0'

MOV AH, 02H ; 设置AH为02H, 准备显示单个字符

INT 21H ; 调用DOS中断21H显示字符'0'

JMP EXIT ; 直接跳转到程序结束

CONVERT_LOOP: ; 十进制转换循环标签

XOR DX, DX ; 清零DX, 准备进行除法

DIV BX ; $AX \div 10$, 商在AX, 余数在DX

PUSH DX ; 余数压栈 (从低位到高位)

INC CX ; 计数压栈的位数加1

OR AX, AX ; 检查商是否为0

JNZ CONVERT_LOOP ; 如果商不为0, 继续循环除法

PRINT_LOOP: ; 显示循环标签

POP DX ; 从栈中弹出余数 (高位先出)

ADD DL, '0' ; 将数字转为ASCII字符

MOV AH, 02H ; 设置AH为02H, 准备显示单个字符

INT 21H ; 调用DOS中断21H显示一位数字

LOOP PRINT_LOOP ; 循环显示所有压栈的位

EXIT: ; 程序结束标签

MOV AH, 4CH ; 设置AH为4CH, 准备退出程序

INT 21H ; 调用DOS中断21H退出程序

CODE_SEG ENDS ; 代码段结束

END MAIN ; 程序结束, 以MAIN为入口

(3) 实验过程

①打开 Masm fow Windows 集成实验环境 2015 后, 将代输入进去并保存文件为 16TO10.asm, 点击运行运行。



图 2-2 Masm fow Windows 集成实验环境 2015 界面打开 16TO10.asm

②分别输入几个 16 进制数字，显示结果如图 2-3，和网页进制转换计算器对比完全一致，如图 2-4 所示

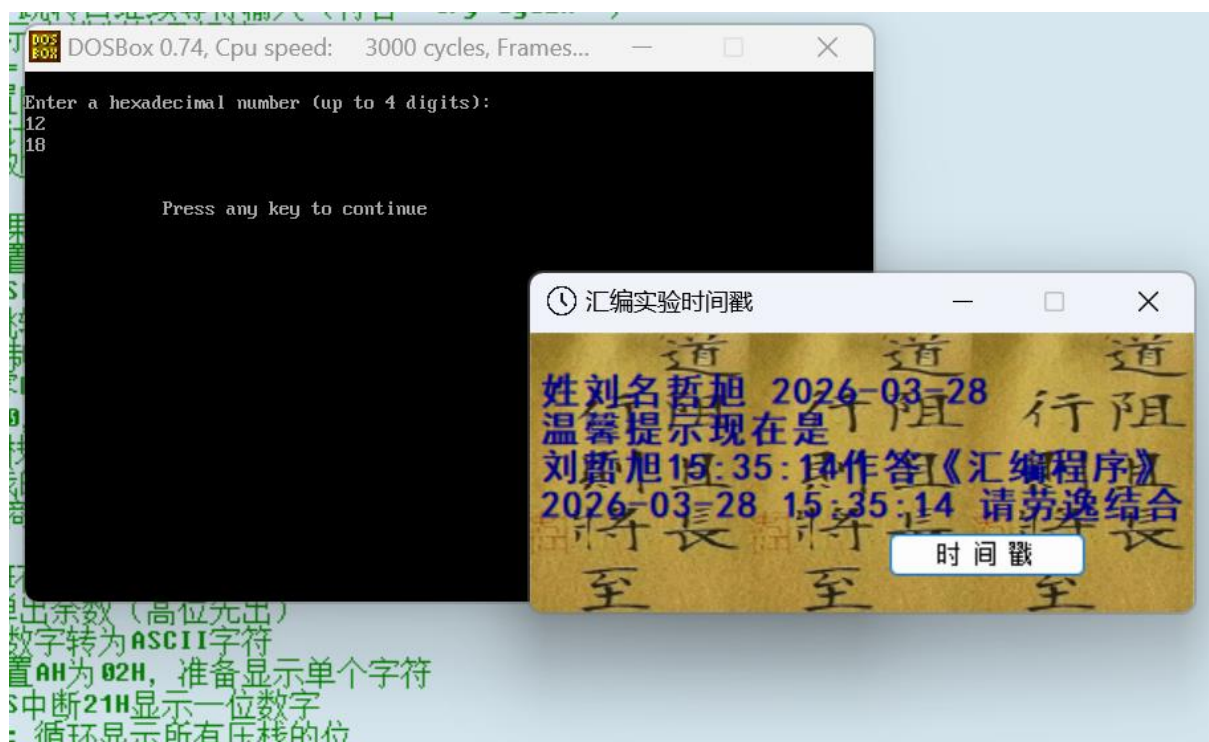


图 2-3(a)

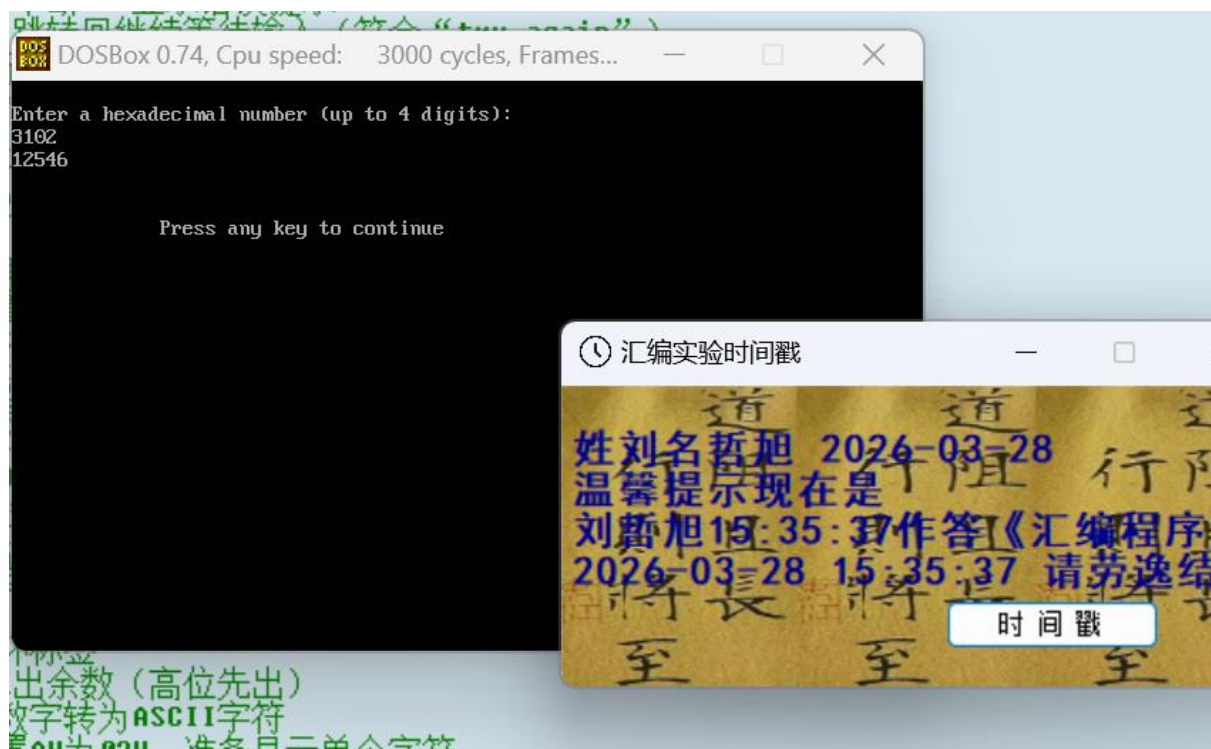


图
2-3(b)

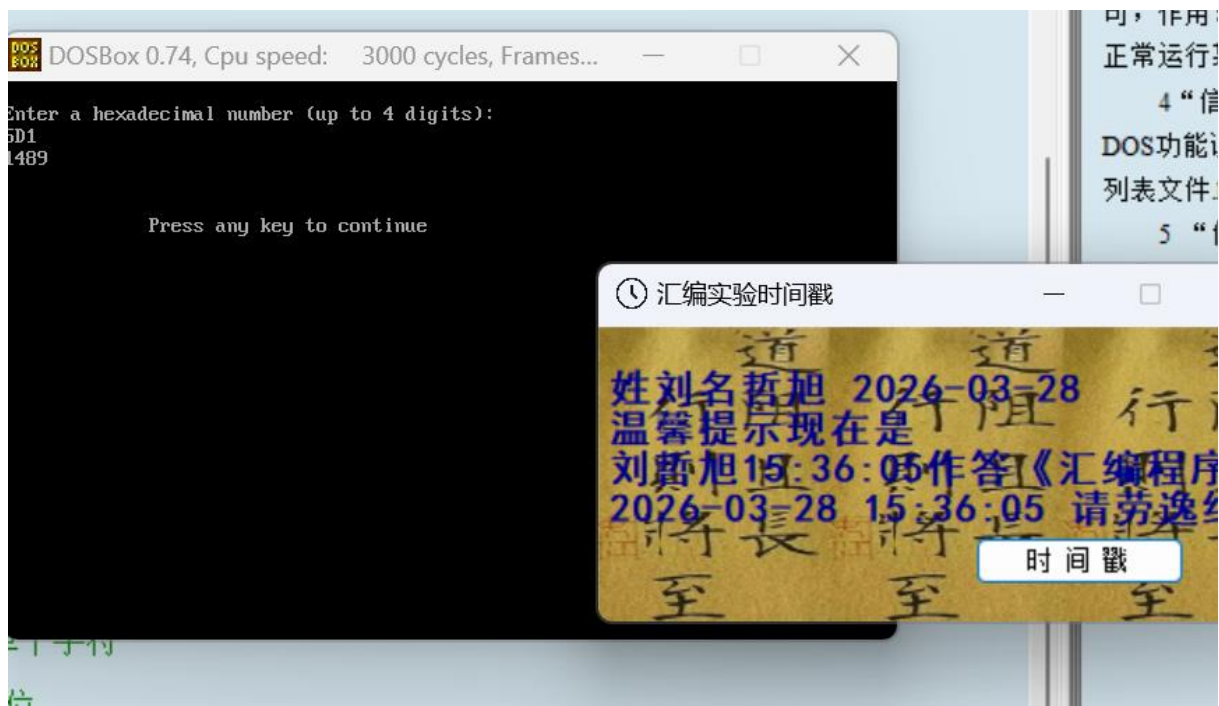


图 2-3(c)

数值进制	☐ 2进制	☐ 8进制	☐ 10进制	☒ 16进制	16进制
输入数值	12				

结果进制	☐ 2进制	☐ 8进制	☒ 10进制	☐ 16进制	10进制
转换结果	18				

数值进制	☐ 2进制	☐ 8进制	☐ 10进制	☒ 16进制	
输入数值	3102				

结果进制	☐ 2进制	☐ 8进制	☒ 10进制	☐ 16进制	
转换结果	12546				

数值进制	☐ 2进制	☐ 8进制	☐ 10进制	☒ 16进制
输入数值	5D1			
结果进制	☐ 2进制	☐ 8进制	☒ 10进制	☐ 16进制
转换结果	1489			

图 2-4

【第三个实验】判断年份是否为闰年

(1) 实验流程图：

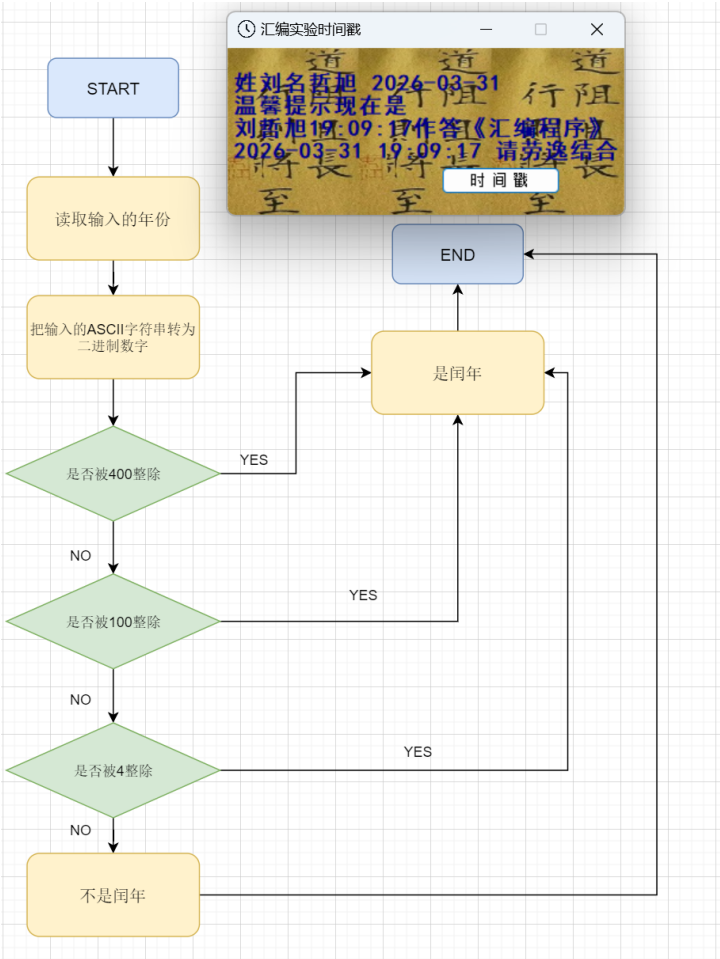


图 3-1：流程图

(2) 实验源代码（粘贴源代码）：

DATAS SEGMENT ; 数据段开始

```

INFON DB 13,10,'Please input a year:',13,10,'$'    ; 定义提示输入年份的字符串, 回车换行后显示“Please input a year:”

Y      DB 13,10,'It is a leap year',13,10,'$'      ; 定义“是闰年”提示字符串, 回车换行后显示“This is a leap year!”

N      DB 13,10,' It is not a leap year',13,10,'$' ; 定义“不是闰年”提示字符串, 回车换行后显示“This is not a leap year!”

W      DW 0                                          ; 定义一个字变量W, 用于存储输入年份转换后的数字, 初始值为0

BUF    DB 8                                          ; 定义输入缓冲区最大可接受的字符长度为8
        DB ?                                         ; 实际输入的字符个数, 由系统自动填充
        DB 8 DUP(0)                                  ; 预留8个字节用于存放输入的字符串内容, 全部初始化为0

DATAS ENDS                                          ; 数据段结束

STACKS SEGMENT STACK                              ; 栈段开始
        DB 32 DUP(0)                                  ; 定义栈空间, 大小为32个字节, 全部初始化为0
STACKS ENDS                                        ; 栈段结束

CODES SEGMENT                                      ; 代码段开始

        ASSUME DS:DATAS, SS:STACKS, CS:CODES      ; 告诉汇编器各段寄存器指向的段

START:                                           ; 程序入口标号

        MOV AX, DATAS                                ; 将数据段DATAS的段地址装入AX寄存器
        MOV DS, AX                                  ; 把AX中的数据段地址传送给DS寄存器, 使DS
    
```

指向数据段

MOV AX, STACKS ; 将栈段STACKS的段地址装入AX寄存器

MOV SS, AX ; 把AX中的栈段地址传送给SS寄存器, 使SS指向

栈段

XOR AX, AX ; 将AX寄存器清零 (异或自身)

; 显示输入提示信息

MOV DX, **OFFSET** INFON ; 将提示字符串INFON的偏移地址装入DX寄存

器

MOV AH, 9 ; 设置AH=9, 调用DOS 9号功能 (显示以\$结尾

的字符串)

INT 21H ; 执行中断, 显示“Please input a year.”

; 从键盘接收用户输入的年份字符串

MOV DX, **OFFSET** BUF ; 将输入缓冲区BUF的偏移地址装入DX寄存器

MOV AH, 0AH ; 设置AH=0AH, 调用DOS 10号功能 (带缓冲

的键盘输入)

INT 21H ; 执行中断, 从键盘读取字符串并存入BUF缓冲区

; 调用子程序将输入的字符串转换为数字

CALL DATACATE ; 调用DATACATE子程序, 把输入的ASCII字符

串转为二进制数字存入W

; 调用子程序判断是否为闰年

CALL IFYEARS ; 调用IFYEARS子程序, 判断年份是否为闰年, 结

果通过进位标志CF返回

```

JC A1                ; 如果进位标志CF=1 (是闰年), 则跳转到A1标号

MOV DX, OFFSET N     ; 不是闰年, 将“不是闰年”字符串的偏移地址装
入DX

MOV AH, 9            ; 设置AH=9, 准备显示字符串

INT 21H              ; 显示“This is not a leap year!”

JMP EXIT             ; 跳转到程序退出处

A1:                  ; 是闰年的处理标号

MOV DX, OFFSET Y     ; 将“是闰年”字符串的偏移地址装入DX

MOV AH, 9            ; 设置AH=9, 准备显示字符串

INT 21H              ; 显示“This is a leap year!”

EXIT:                ; 程序退出标号

MOV AH, 4CH           ; 设置AH=4CH, 调用DOS 4CH号功能 (终止程
序)

INT 21H              ; 执行中断, 退出程序并返回DOS

                        ;子程序: 字符串转换为数字

DATACATE PROC NEAR   ; 定义近过程子程序DATACATE (字符串转数
字)

PUSH CX              ; 将CX寄存器压入栈中备份 (保护现场)

PUSH SI              ; 将SI寄存器压入栈中备份

PUSH BX              ; 将BX寄存器压入栈中备份

PUSH DX              ; 将DX寄存器压入栈中备份

XOR AX, AX           ; 将AX清零, 作为累加结果使用

```

<code>MOV [W], AX</code>	; 把W变量也清零, 准备存放最终年份数字
<code>MOV CL, [BUF+1]</code>	; 将实际输入的字符个数取出, 放入CL中 (用于
循环控制)	
<code>MOV SI, OFFSET BUF+2</code>	; SI指向输入字符串的第一个有效字符 (BUF+2
位置)	
<code>CONVERT_LOOP:</code>	; 字符串转数字循环标号
<code>MOV BL, [SI]</code>	; 从SI指向的内存单元取出一个字符 (ASCII码)
<code>SUB BL, 30H</code>	; 把ASCII码减去'0'的ASCII值 (30H), 转换为对
应的十进制数字0-9	
<code>MOV BH, 0</code>	; 将BH清零, 使BX成为完整的16位数字
<code>MOV DX, 10</code>	; 把10放入DX, 准备做乘10运算
<code>MUL DX</code>	; AX = AX * 10 (把之前的结果左移一位, 相当于
乘10)	
<code>ADD AX, BX</code>	; 把当前数字加到AX中, 实现 result = result*10
+ digit	
<code>INC SI</code>	; SI加1, 指向下一个字符
<code>DEC CL</code>	; 输入长度计数器CL减1
<code>JNZ CONVERT_LOOP</code>	; 如果CL不为0, 继续循环处理下一个字符
<code>MOV [W], AX</code>	; 转换完成, 把最终结果存入变量W中
<code>POP DX</code>	; 从栈中弹出DX, 恢复现场
<code>POP BX</code>	; 从栈中弹出BX, 恢复现场
<code>POP SI</code>	; 从栈中弹出SI, 恢复现场

```

POP CX                ; 从栈中弹出CX, 恢复现场

RET                  ; 子程序返回

DATACATE ENDP        ; DATACATE子程序结束

; 子程序: 判断是否为闰年

IFYEARS PROC NEAR    ; 定义近过程子程序IFYEARS (判断闰年)

    PUSH BX          ; 将BX寄存器压入栈中备份
    PUSH CX          ; 将CX寄存器压入栈中备份
    PUSH DX          ; 将DX寄存器压入栈中备份
    MOV AX, [W]       ; 把转换后的年份数字从W中取出, 放入AX寄存
器
    MOV CX, AX        ; 将年份备份到CX寄存器中, 供后续多次使用
    ; 第一步: 判断年份能否被400整除
    XOR DX, DX        ; 把DX清零, 防止除法时出现溢出错误
    MOV BX, 400       ; 把400放入BX, 作为除数
    DIV BX            ; AX除以BX, 商在AX, 余数在DX
    CMP DX, 0         ; 判断余数是否为0
    JZ IS_LEAP        ; 如果余数为0 (能被400整除), 则一定是闰年,
跳转到IS_LEAP
    ; 第二步: 判断年份能否被100整除
    MOV AX, CX        ; 恢复年份到AX寄存器
    XOR DX, DX        ; 再次清零DX
    MOV BX, 100       ; 把100放入BX, 作为除数

```

```

DIV BX                ; AX除以100

CMP DX, 0             ; 判断余数是否为0

JZ NOT_LEAP          ; 如果能被100整除但不能被400整除, 则不是闰
年

; 第三步: 判断年份能否被4整除

MOV AX, CX            ; 再次恢复年份到AX

XOR DX, DX            ; 清零DX

MOV BX, 4             ; 把4放入BX, 作为除数

DIV BX                ; AX除以4

CMP DX, 0             ; 判断余数是否为0

JZ IS_LEAP            ; 如果能被4整除, 则是闰年

NOT_LEAP:             ; 不是闰年的处理

CLC                   ; 清进位标志CF=0, 表示不是闰年

JMP END_IF            ; 跳转到子程序结束部分

IS_LEAP:              ; 是闰年的处理

STC                   ; 置进位标志CF=1, 表示是闰年

END_IF:               ; 判断结束标号

POP DX                ; 恢复DX寄存器

POP CX                ; 恢复CX寄存器

POP BX                ; 恢复BX寄存器

RET                   ; 子程序返回

IFYEARS ENDP          ; IFYEARS子程序结束

```

CODES ENDS

; 代码段结束

END START

; 整个程序结束, 指定入口为START

(3) 实验过程:

①打开 Masm fow Windows 集成实验环境 2015 后,输入代码并保存为 Leapyear.asm, 点击运行。

```

001 DATAS SEGMENT                ; 数据段开始
002     INFON DB 13,10,'Please input a year:',13,10,'$' ; 定义提示输入年份的字符串,回车换行后显示“
003     Y DB 13,10,'It is a leap year',13,10,'$' ; 定义“是闰年”提示字符串,回车换行后显示“Thi
004     N DB 13,10,'It is not a leap year',13,10,'$' ; 定义“不是闰年”提示字符串,回车换行后显示“T
005     W DW 0 ; 定义一个字变量W,用于存储输入年份转换后的数字,初始值为0
006     BUF DB 8 ; 定义输入缓冲区最大可接受的字符长度为8
007     DB ? ; 实际输入的字符个数,由系统自动填充
008     DB 8 DUP(0) ; 预留8个字节用于存放输入的字符串内容,全部初始化为0
009 DATAS ENDS                    ; 数据段结束
010
011 STACKS SEGMENT STACK         ; 栈段开始
012     DB 32 DUP(0) ; 定义栈空间,大小为32个字节,全部初始化为0
013 STACKS ENDS                  ; 栈段结束
014
015 CODES SEGMENT                ; 代码段开始
016     ASSUME DS:DATAS, SS:STACKS, CS:CODES ; 告诉汇编器各段寄存器指向的段
017
018 START:                        ; 程序入口标号
019     MOV AX, DATAS ; 将数据段DATAS的段地址装入AX寄存器
020     MOV DS, AX ; 把AX中的数据段地址传送给DS寄存器,使DS指向数据段
021     MOV AX, STACKS ; 将栈段STACKS的段地址装入AX寄存器
022     MOV SS, AX ; 把AX中的栈段地址传送给SS寄存器,使SS指向栈段
023     XOR AX, AX ; 将AX寄存器清零(异或自身)
024
025 ; 显示输入提示信息
026     MOV DX, OFFSET INFON ; 将提示字符串INFON的偏移地址装入DX寄存器
027     MOV AH, 9 ; 设置AH=9,调用DOS 9号功能
028     INT 21H ; 执行中断,显示“Please input a year:”
029
030 ; 从键盘接收用户输入的年份字符串
031     MOV DX, OFFSET BUF ; 将输入缓冲区BUF的偏移地址装入DX寄存器
032     MOV AH, 0AH ; 设置AH=0AH,调用DOS 10号功能
033     INT 21H ; 执行中断,从键盘读取字符串
034
035 ; 调用子程序将输入的字符串转换为数字
036     CALL DATACATE ; 调用DATACATE子程序,把输入的ASCII字符串转为二进制数字存入W
037
038 ; 调用子程序判断是否为闰年
039     CALL IFYEARS ; 调用IFYEARS子程序,判断年份是否为闰年,结果通过进位标志CF返回
040
041     JC A1 ; 如果进位标志CF=1(是闰年),则跳转到A1标号
042     MOV DX, OFFSET N ; 不是闰年,将“不是闰年”字符串的偏移地址装入DX
043     MOV AH, 9 ; 设置AH=9,准备显示字符串
    
```

图 3-2: Masm fow Windows 集成实验环境 2015 界面打开 Leapyear.asm



图 3-3：输入不同年份判断是否为闰年

【第四个实验】：两位数加法

(1) 实验流程图：

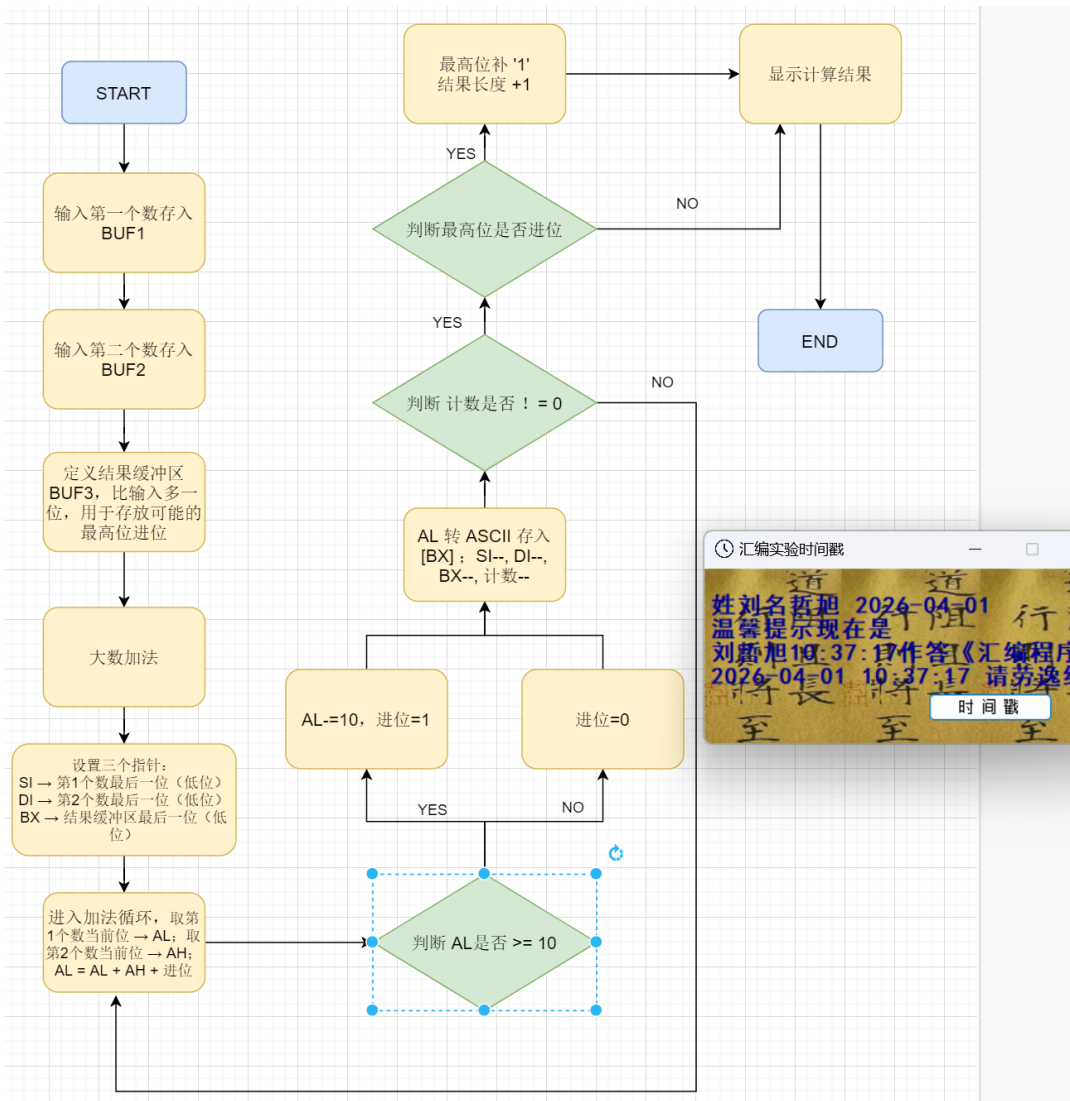


图 4-1：流程图

(2) 实验源代码（粘贴源代码）：

DATAS SEGMENT

MSG1 DB 13,10,'Enter the first number: \$' ; 定义第一个数的输入提示字符串, 回车换行后显示提示信息

MSG2 DB 13,10,'Enter the second number: \$' ; 定义第二个数的输入提示字符串, 回车换行后显示提示信息

```

MSG3 DB 13,10,'The sum is: $'           ; 定义结果输出提示字符串, 回车换行后显示"The sum is: "

BUFFER1 DB 200                          ; 定义第一个数输入缓冲区, 最大可输入199个字符
        DB ?                            ; 用于存放实际输入的字符个数, 由系统自动填充
        DB 200 DUP(?)                   ; 预留200个字节用于存放用户输入的第二个数字字符串

BUFFER2 DB 200                          ; 定义第二个数输入缓冲区, 最大可输入199个字符
        DB ?                            ; 用于存放实际输入的字符个数, 由系统自动填充
        DB 200 DUP(?)                   ; 预留200个字节用于存放用户输入的第二个数字字符串

BUFFER3 DB 201                          ; 定义结果缓冲区, 比输入多1位, 用于存放可能的最高位进位 (如999+1=1000)
        DB ?                            ; 用于存放最终结果的实际字符个数
        DB 201 DUP(?)                   ; 预留201个字节用于存放计算后的结果字符串

DATAS ENDS

STACKS SEGMENT STACK

```

```

        DW 128 DUP(0)                ; 定义栈段, 分配128个字 (256字节) 的栈
空间, 并全部初始化为0

STACKS ENDS

CODES SEGMENT

        ASSUME DS:DATAS, SS:STACKS, CS:CODES    ; 告诉汇编器: DS指向数据段,
SS指向栈段, CS指向代码段

MAIN:

        MOV AX, DATAS                ; 将数据段DATAS的段地址装入AX寄存器

        MOV DS, AX                  ; 把AX中的段地址传送给DS, 使DS指向数据
段

        MOV AX, STACKS              ; 将栈段STACKS的段地址装入AX寄存器

        MOV SS, AX                  ; 把AX中的段地址传送给SS, 使SS指向栈段

        ; 输入第一个大数

        MOV DX, OFFSET MSG1         ; 将第一个输入提示字符串的偏移地址装
入DX寄存器

        MOV AH, 9                   ; 设置AH=9, 调用DOS 9号功能 (显示以$结
尾的字符串)

        INT 21H                     ; 执行中断, 屏幕上显示“Enter the first
number: ”

        MOV DX, OFFSET BUFFER1      ; 将BUFFER1缓冲区的偏移地址装入DX

        MOV AH, 10                   ; 设置AH=10, 调用DOS 10号功能 (带缓冲
的键盘字符串输入)
    
```

```

INT 21H                                ; 从键盘接收用户输入的第二个数字字符串并
存入BUFFER

; 输入第二个大数

MOV DX, OFFSET MSG2                    ; 将第二个输入提示字符串的偏移地址装
入DX寄存器

MOV AH, 9                              ; 设置AH=9, 准备显示字符串

INT 21H                                ; 执行中断, 屏幕上显示“Enter the second
number.”

MOV DX, OFFSET BUFFER2                  ; 将BUFFER2缓冲区的偏移地址装入DX

MOV AH, 10                              ; 设置AH=10, 准备接收键盘输入

INT 21H                                ; 从键盘接收用户输入的第二个数字字符串并
存入BUFFER2

; 显示结果提示

MOV DX, OFFSET MSG3                    ; 将结果提示字符串的偏移地址装入DX

MOV AH, 9                              ; 设置AH=9, 准备显示字符串

INT 21H                                ; 执行中断, 屏幕上显示“The sum is: ”

CALL PROCESS_DATA                       ; 调用大数加法处理子程序, 完成两个大数
的相加

CALL DISPLAY_RESULT                     ; 调用结果显示子程序, 将计算结果输出到
屏幕

MOV AH, 4CH                            ; 设置AH=4CH, 调用DOS终止程序功能

INT 21H                                ; 执行中断, 退出程序并返回到DOS

```

；子程序：大数加法处理（从低位到高位逐位相加，正确处理进位和不同长度）

PROCESS_DATA PROC NEAR

```

PUSH AX                ; 将AX寄存器压栈，保护现场
PUSH BX                ; 将BX寄存器压栈，保护现场
PUSH CX                ; 将CX寄存器压栈，保护现场
PUSH DX                ; 将DX寄存器压栈，保护现场
PUSH SI                ; 将SI寄存器压栈，保护现场
PUSH DI                ; 将DI寄存器压栈，保护现场

; 确定循环次数：取两个数中较长的长度

MOV CL, [BUFFER1+1]    ; CL = 第一个数的实际输入长度
MOV CH, [BUFFER2+1]    ; CH = 第二个数的实际输入长度
CMP CL, CH              ; 比较两个数的长度
JAE SETLEN              ; 如果CL >= CH，则跳转到SETLEN
MOV CL, CH              ; 否则把较长的长度放入CL
    
```

SETLEN:

```

MOV BYTE PTR [BUFFER3+1], CL    ; 把当前最大长度暂存到结果缓冲区的长度
    
```

字段

```

; 计算第一个数最后一位的地址（低位）

MOV SI, OFFSET BUFFER1 + 2    ; SI指向BUFFER1中第一个字符的位置
MOV DL, CL                    ; 用DL暂存长度（避免直接用CL加SI）
XOR DH, DH                    ; DH清零，使DX成为完整的字
ADD SI, DX                     ; SI = BUFFER1起始地址 + 长度
    
```

DEC SI ; SI指向最后一个有效字符（低位）

; 计算第二个数最后一位的地址（低位）

MOV DI, OFFSET BUFFER2 + 2 ; DI指向BUFFER2中第一个字符的位置

ADD DI, DX ; DI = BUFFER2起始地址 + 长度

DEC DI ; DI指向最后一个有效字符（低位）

; 计算结果缓冲区最后一位的地址（低位开始存放）

MOV BX, OFFSET BUFFER3 + 2 ; BX指向BUFFER3中第一个字符的位置

ADD BX, DX ; BX = BUFFER3起始地址 + 长度

DEC BX ; BX指向结果缓冲区的最后一位

MOV DH, 0 ; DH作为进位标志，初始为0

ADDLOOP:

MOV AL, 0 ; AL清零，默认第一个数当前位为0

CMP CL, [BUFFER1+1] ; 判断是否超过第一个数的实际长度

JA SKIP1 ; 如果超过，则跳过取数（保持AL=0）

MOV AL, [SI] ; 取出第一个数当前位的ASCII码

SUB AL, 30H ; 把ASCII码转换为实际数字（0-9）

SKIP1:

MOV AH, 0 ; AH清零，默认第二个数当前位为0

CMP CL, [BUFFER2+1] ; 判断是否超过第二个数的实际长度

JA SKIP2 ; 如果超过，则跳过取数（保持AH=0）

MOV AH, [DI] ; 取出第二个数当前位的ASCII码

SUB AH, 30H ; 把ASCII码转换为实际数字（0-9）

SKIP2:

<code>ADD AL, AH</code>	；把两个当前位的数字相加，结果在AL中
<code>ADD AL, DH</code>	；加上上一位产生的进位
<code>MOV DH, 0</code>	；先把进位标志清零
<code>CMP AL, 10</code>	；判断当前位相加结果是否大于等于10
<code>JB NOCARRY</code>	；如果小于10，则不需要进位
<code>SUB AL, 10</code>	；大于等于10时，减去10
<code>MOV DH, 1</code>	；设置进位标志为1

NOCARRY:

<code>ADD AL, 30H</code>	；把计算后的数字转回ASCII码
<code>MOV [BX], AL</code>	；把结果存入结果缓冲区当前位
<code>DEC SI</code>	；SI指向第一个数的前一位
<code>DEC DI</code>	；DI指向第二个数的前一位
<code>DEC BX</code>	；BX指向结果缓冲区的前一位
<code>DEC CL</code>	；循环计数器减1
<code>JNZ ADDLOOP</code>	；如果还没有处理完所有位，则继续循环
；处理最高位可能产生的进位	
<code>CMP DH, 0</code>	；检查是否有最终进位
<code>JE NOFINAL</code>	；如果没有进位，则跳转
<code>MOV BYTE PTR [BX], '1'</code>	；有进位时，在最高位补上字符'1'
<code>INC BYTE PTR [BUFFER3+1]</code>	；结果的总长度加1

NOFINAL:

```

POP DI                ; 恢复DI寄存器
POP SI                ; 恢复SI寄存器
POP DX                ; 恢复DX寄存器
POP CX                ; 恢复CX寄存器
POP BX                ; 恢复BX寄存器
POP AX                ; 恢复AX寄存器
RET                  ; 子程序返回

```

PROCESS_DATA END

; 子程序：显示最终计算结果（从高位到低位正常显示）

DISPLAY_RESULT PROC NEAR

```

PUSH AX              ; 保护AX寄存器
PUSH BX              ; 保护BX寄存器
PUSH CX              ; 保护CX寄存器
PUSH DX              ; 保护DX寄存器
MOV CL, [BUFFER3+1] ; CL = 结果字符串的实际长度
MOV CH, 0            ; CH清零，使CX成为完整的字计数器
MOV BX, OFFSET BUFFER3 + 2 ; BX指向结果缓冲区的第一个字符（最高位）

```

DISPLOOP:

```

MOV DL, [BX]         ; 把当前一位字符取出到DL
MOV AH, 2            ; 设置AH=2，调用DOS显示单个字符功能
INT 21H              ; 执行中断，在屏幕上显示一位数字
INC BX               ; BX指向结果的下一位

```

LOOP DISPLOOP ; 按照长度循环显示所有位

POP DX ; 恢复DX寄存器

POP CX ; 恢复CX寄存器

POP BX ; 恢复BX寄存器

POP AX ; 恢复AX寄存器

RET ; 子程序返回

DISPLAY_RESULT ENDP

CODES ENDS

END MAIN ; 整个程序结束, 指定程序入口为MAIN标号

(3) 实验过程

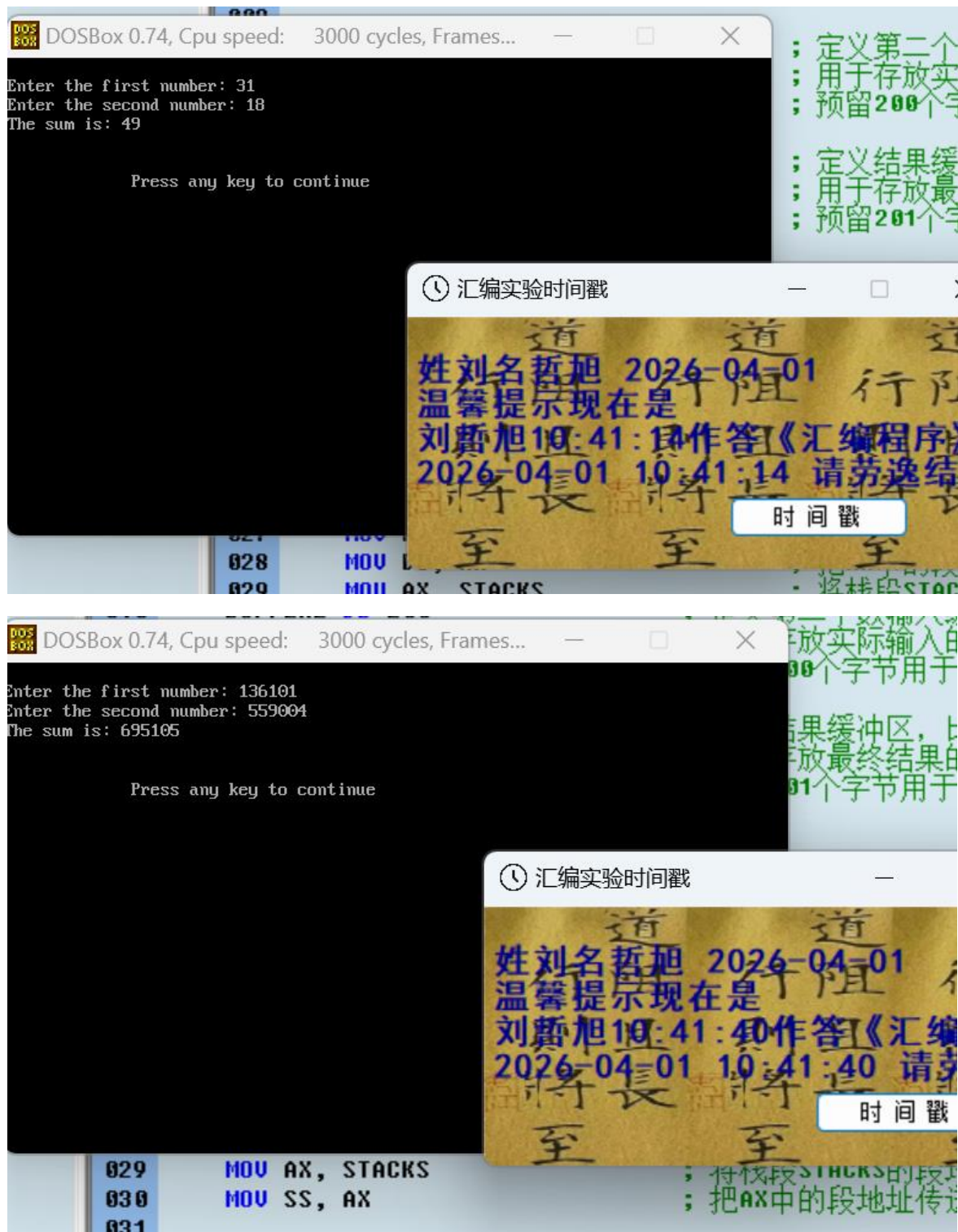
①打开 Masm fow Windows 集成实验环境 2015 后, 输入代码并保存为 Leapyear.asm, 点击运行, 如图 4-2 所示

```

001 DATAS SEGMENT
002     MSG1 DB 13,10,'Enter the first number: $' ; 定义第一个数的输入提示字符串, 回车换行后显示提示
003     MSG2 DB 13,10,'Enter the second number: $' ; 定义第二个数的输入提示字符串, 回车换行后显示提示
004     MSG3 DB 13,10,'The sum is: $' ; 定义结果输出提示字符串, 回车换行后显示
005
006     BUFFER1 DB 200 ; 定义第一个数输入缓冲区, 最大可输入199个字符
007             DB ? ; 用于存放实际输入的字符个数, 由系统自动填充
008             DB 200 DUP(?) ; 预留200个字节用于存放用户输入的第二个数字字符串
009
010     BUFFER2 DB 200 ; 定义第二个数输入缓冲区, 最大可输入199个字符
011             DB ? ; 用于存放实际输入的字符个数, 由系统自动填充
012             DB 200 DUP(?) ; 预留200个字节用于存放用户输入的第二个数字字符串
013
014     BUFFER3 DB 201 ; 定义结果缓冲区, 比输入多1位, 用于存放可能的最高位进位(如999+1=
015             DB ? ; 用于存放最终结果的实际字符个数
016             DB 201 DUP(?) ; 预留201个字节用于存放计算后的结果字符串
017 DATAS ENDS
018
019 STACKS SEGMENT STACK
020     DW 128 DUP(0) ; 定义栈段, 分配128个字(256字节)的栈空间, 并全部初始化为0
021 STACKS ENDS
022
023 CODES SEGMENT
024     ASSUME DS:DATAS, SS:STACKS, CS:CODES ; 告诉汇编器: DS指向数据段, SS指向栈段, CS指向代码段
025
026 MAIN:
027     MOV AX, DATAS ; 将数据段DATAS的段地址装入AX寄存器
028     MOV DS, AX ; 把AX中的段地址传送给DS
029     MOV AX, STACKS ; 将栈段STACKS的段地址装入AX寄存器
030     MOV SS, AX ; 把AX中的段地址传送给SS
031
032     ; 输入第一个大数
033     MOV DX, OFFSET MSG1 ; 将第一个输入提示字符串的偏移地址装入DX寄存器
034     MOV AH, 9 ; 设置AH=9, 调用DOS 9号中断
035     INT 21H ; 执行中断, 屏幕上显示"Enter the first number: "
036
037     MOV DX, OFFSET BUFFER1 ; 将BUFFER1缓冲区的偏移地址装入DX寄存器
038     MOV AH, 10 ; 设置AH=10, 调用DOS 10号中断
039     INT 21H ; 从键盘接收用户输入的字符并放入BUFFER1
040
041     ; 输入第二个大数
042     MOV DX, OFFSET MSG2 ; 将第二个输入提示字符串的偏移地址装入DX寄存器
043     MOV AH, 9 ; 设置AH=9, 调用DOS 9号中断
    
```

图 4-2: 流程图

②在 DOSBOX 界面输入两个相同尾数的加数，多次实验运行，实验结果如图 4-2 所示



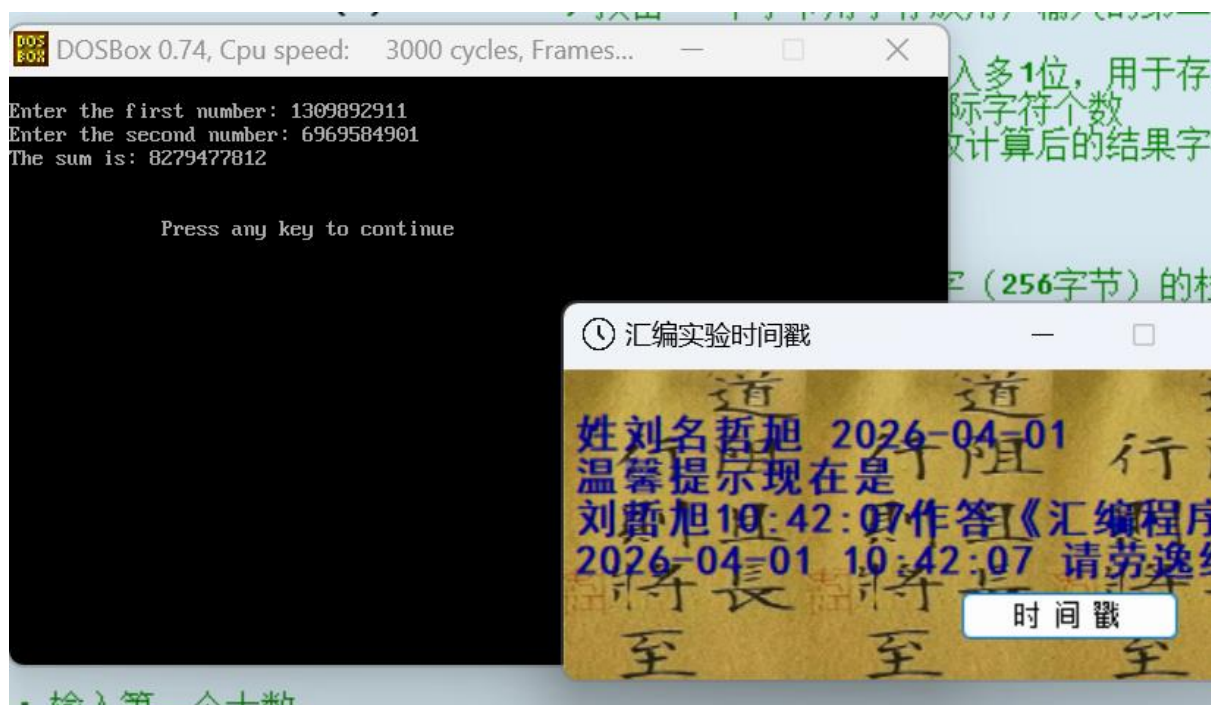


图 4-3：输入相同位数的两个加数

③处理相同位数加法时可以正常运行，但输入不同位数时发现处理不同位数的加法时结果出现错误，如图 4-4 所示，原因在后续会补充。

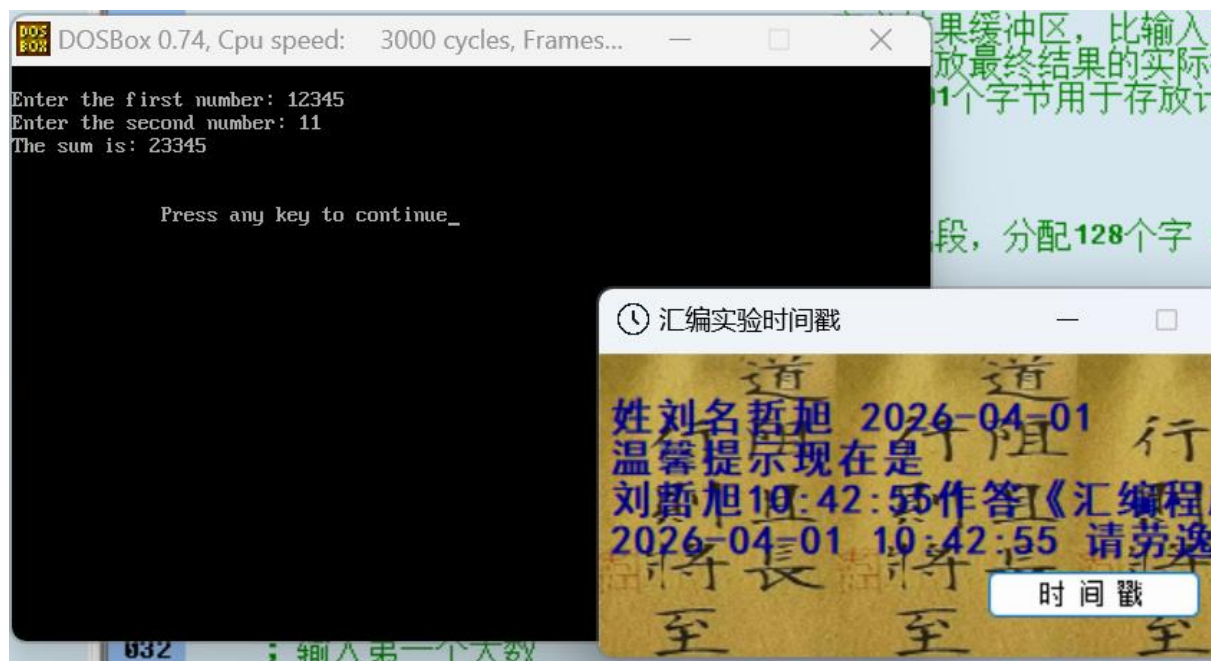


图 4-4：输入不同位数的两个加数

【第五个实验】My heart beats with yours!

将 MyHeart.asm 保存到 masm5 文件夹中，通过 DOSBOX 运行，结果如图 5 所示：

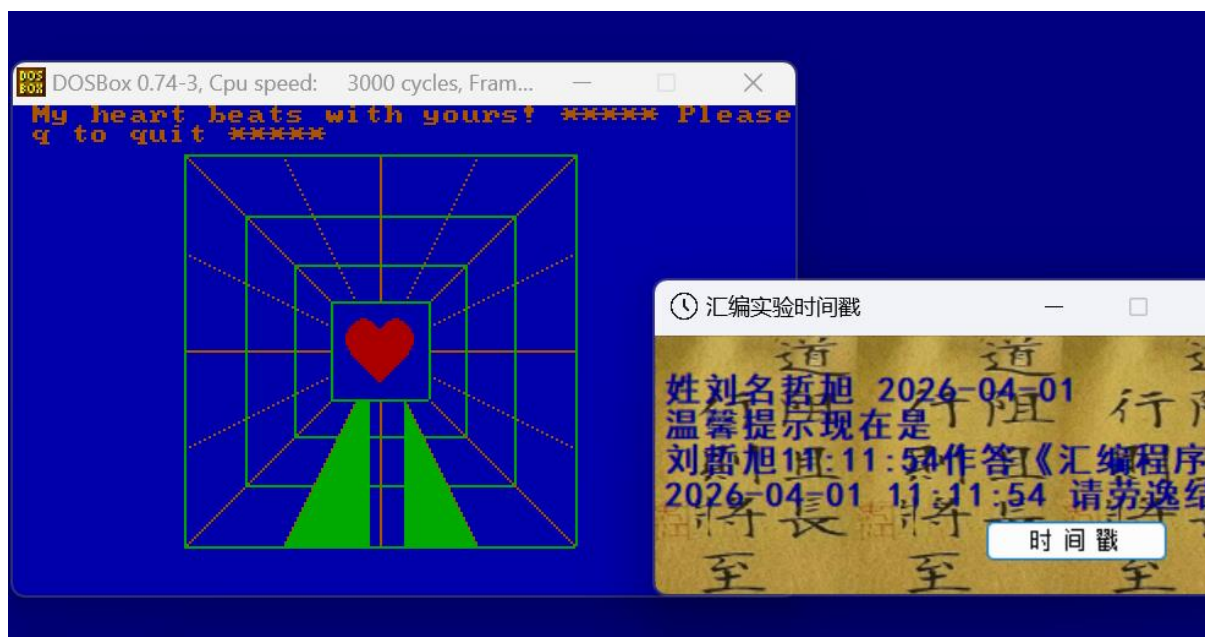


图 5: My heart beats with yours!

【实验心得】

1.在实验四的测试过程中发现，当输入两个位数不同的十进制数进行加法运算时，程序输出结果出现异常，表现为结果错位、出现乱码、少位或多出无关字符等现象。经过查阅资料，主要原因如下：

本程序的核心加法逻辑位于 `PROCESS_DATA` 子程序中。该子程序试图通过从低位开始逐位相加的方式实现大数加法，但存在以下几个关键缺陷：

首先，低位对齐处理不正确。多位数加法必须严格从最低位（个位）开始对齐进行运算，当两个加数位数不同时，位数较短的数的高位应当自动补 0。但原代码在计算指针 `SI` 和 `DI` 的初始位置时，采用简单的长度叠加方式，没有正确处理两个缓冲区长度差异，导致在处理较高位时，取数逻辑混乱，无法为较短的数提供正确的 0 值补位，从而造成计算结果错位。

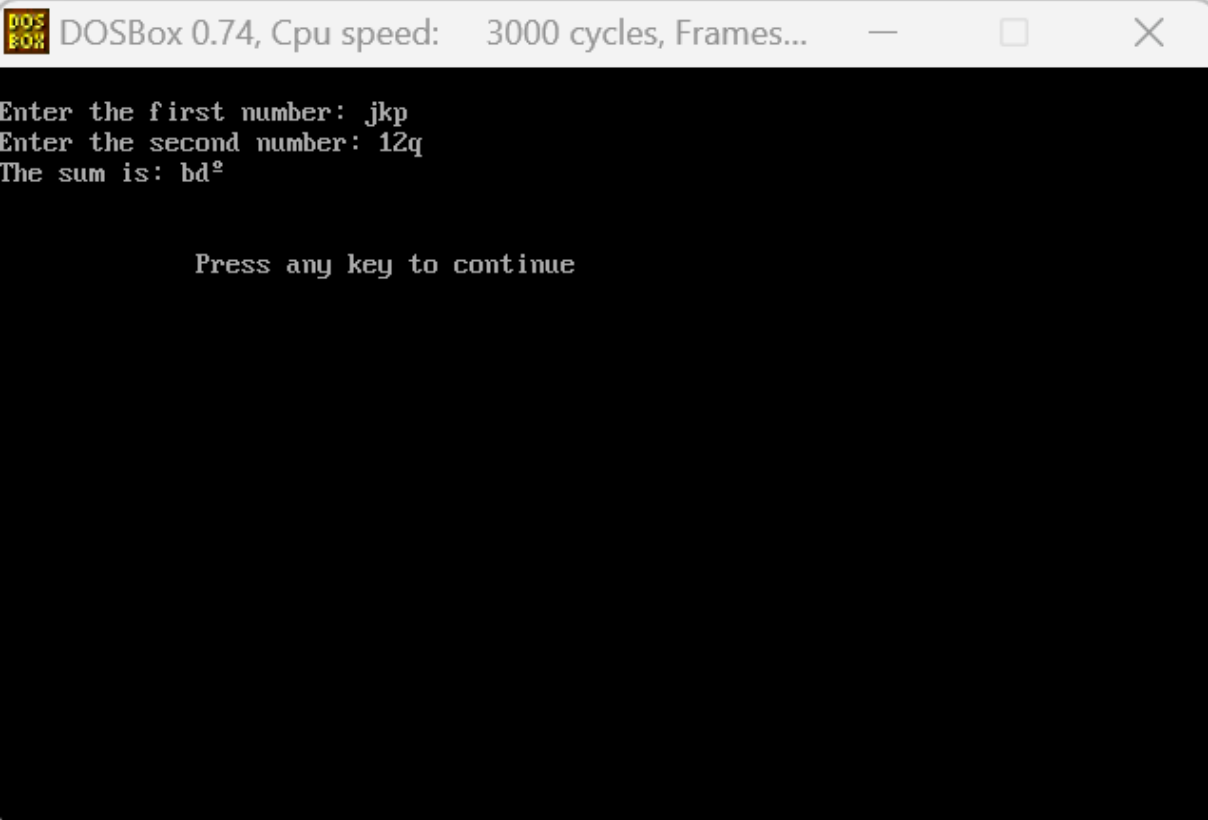
其次，指针移动与结果存放逻辑存在冲突。程序在循环中同时对 `SI`、`DI`、`BX` 三个指针进行递减操作，但三者的初始定位和移动步调没有严格保持一致。在不同位数的情况下，这种指针管理方式容易导致结果被存放在错误的位置，或者覆盖了 `BUFFER3` 中的无效数据区域，最终使显示结果出现乱码或缺失有效数字。

第三，进位处理机制不完善。虽然程序设置了 `DH` 作为进位标志，但在最高位进位的处理时机和位置判断上存在缺陷。当最高位产生进位时（例如 $999 + 1 = 1000$ ），新产生的最高位未能正确插入到结果字符串的正确位置，导致结果长度管理错误，进一步加剧了输出异常。

此外，结果缓冲区 `BUFFER3` 的长度字段更新不准确。原代码仅在最高位进位时才

增加长度计数，而在正常循环过程中对有效结果长度的累计机制不健全，导致 DISPLAY_RESULT 子程序在显示时无法准确确定需要输出的字符范围，从而输出多余的垃圾字符或截断有效结果。

2.实验四中，我好奇如果两次输入均包含字母，输出结果会是怎样。最终结果是一个很奇怪的组合，经过查阅资料得知，主要原因是程序没有对输入内容进行有效性检查和数字转换处理。



```

DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
Enter the first number: jk
Enter the second number: 12q
The sum is: bd

Press any key to continue
    
```

本程序使用 INT 21H 的 AH=10 (0AH) 功能进行带缓冲的键盘字符串输入，该功能会把用户输入的任意字符（包括字母、数字、符号等）原样存入 BUFFER1 和 BUFFER2 的缓冲区中，并不区分输入的是数字还是字母。程序后续在 PROCESS_DATA 子程序中，直接把缓冲区中的字符当作数字进行处理（执行 SUB AL, 30H 等操作，将其视为十进制数字的 ASCII 码）。

当输入字母时（如 'j' 的 ASCII 码为 6AH, 'k' 为 6BH, 'p' 为 70H），执行 SUB AL, 30H 后得到的是一个没有意义的大数值（远大于 9），而不是有效的 0~9 数字。这些无效数值进入加法循环后，会产生错误的相加结果和进位，最终导致输出的字符变成乱码或无意义符号

3.关于第二个实验，我套用了输入循环、字符转换和按位除法显示逻辑，以为只要把 SHL 左移 4 位、DIV 除法和 ASCII 转换串起来就能搞定。没想到实际运行后，无论输入什么，最后屏幕上总是固定显示“87”或“8H”这样的奇怪字母。反复检查代码和查阅资料后才发现，问题出在显示部分对寄存器的使用太粗心：每次 DIV 之后余数保存在

DX，却没有及时把 DX 正确传回 AX（或 CX）供下一位使用，导致低位计算时数据被破坏，尤其输入位数少于 5 位时，CL 和 AL 的混用更是雪上加霜。这次 bug 让我真正理解了汇编语言寄存器的重要性，一条指令的疏忽就会让整个程序输出完全错误

【课堂补充】

一、8086 微处理器在响应中断（无论是硬件中断、软件中断还是内部中断）时，CPU 会自动完成一系列操作，通常可以概括为以下四个主要阶段：

1. 保护现场（保存当前状态）

CPU 首先将标志寄存器（Flags）、代码段寄存器（CS）和指令指针（IP）依次压入栈中保存。这样做的目的是为了在中断处理完成后，能够正确返回到被中断的程序继续执行。

2. 关闭中断（禁用进一步中断）

CPU 自动清除中断允许标志（IF）和单步标志（TF），防止在处理当前中断的过程中被其他中断打断（避免嵌套中断导致混乱）。

3. 获取中断向量（查找中断服务程序入口地址）

CPU 根据中断类型号（0~255）计算中断向量表中的地址（类型号 $\times 4$ ），然后从中断向量表（通常位于内存 00000H~003FFH）中取出中断服务程序的入口地址：低字送入 IP，高字送入 CS。

4. 转入中断服务程序执行

CPU 将控制权转移到中断服务程序（ISR）的起始地址，开始执行中断处理代码。中断处理完成后，通常通过 IRET 指令返回主程序（IRET 会自动恢复 Flags、CS、IP）。

二、ORG

ORG 伪指令的作用

ORG 是 Origin 的缩写，它是一个伪指令，用于指定程序或数据在内存中的起始偏移地址。例如 ORG 1000H，表示从这一行开始，后续的所有指令或数据在内存中的偏移地址从 1000H 开始计算。ORG 只影响汇编器计算地址，并不生成实际的机器码。

ORG 伪指令的使用特点

ORG 常用于精确定位代码或数据在内存中的位置，例如在 COM 文件中通常使用 ORG 100H。它可以出现在代码段或数据段中，帮助程序员控制程序加载后的内存布局，但不会改变 CPU 实际执行时的物理地址。

PTR 操作符的含义

PTR 是 Pointer 的缩写，称为类型指针操作符。其主要作用是强制指定内存操作数的类型（大小），以解决汇编器无法自动判断操作数宽度时的的问题。

WORD PTR 的具体用法

WORD PTR 表示将后面的内存操作数强制按“字”（2 字节，16 位）来处理。例如 MOV

WORD PTR [1000H], 1234H, 就是明确告诉汇编器要把 1234H 作为一个字写入内存地址 1000H 处。如果不使用 PTR, 汇编器在某些情况下可能会报错或产生错误代码。

除了 WORD PTR 之外, 常用的还有 BYTE PTR (按字节处理)、DWORD PTR (按双字/4 字节处理) 等。PTR 操作符在对内存单元进行操作, 尤其是通过寄存器间接寻址 (如 [BX]、[SI]) 时非常重要。

ORG 与 PTR 的区别 ORG 是用来指定汇编起始偏移地址的定位伪指令, 而 PTR 是用来明确内存操作数数据类型的强制转换操作符。两者虽然都属于伪指令, 但作用完全不同: ORG 影响地址计算, PTR 影响指令的汇编方式。



知识点

1. ASSUME 伪指令, 它通知汇编程序 (编译器) 代码段、数据段、附加段以及堆栈段选择什么名字。没有 ASSUME 伪指令时, 汇编程序假设不分段, 并自动把段超越前缀用于所有寻址存储器数据的伪指令。

ASSUME 语句只能用于完整的段定义, ASSUME 不是汇编指令, 仅仅是写给编译软件的。是让编译器知道你的程序的各个段的名称, 并不生成机器码。

ASSUME 伪指令是指明变量与段寄存器的联系, 比如 ASSUME DS:DATAS, 它是告诉编译器以后所有在 DATAS 段中定义的变量寻址时, 使用 DS 作为段地址, 但是它不对程序作任何事 (因为是伪指令), 你必须自己对 DS 赋值。所以, 要让机器知道实际的段地址, 还需要 MOV AX, DATAS MOV DS, AX 告诉 CPU 数据段的地址。

2. Debug 中 P、T、G 指令的区别

在 DOS 的 DEBUG 调试工具中, P、T、G 是三种常用的单步/断点执行命令, 它们的功能和行为有明显区别:

(1) T 指令 (Trace)

作用: 单步执行指令。

特点: 每按一次 T, 只执行一条机器指令。

特别之处: 会进入子程序内部 (CALL 指令会跟进去, 一步一步执行子程序里的每一条指令)。

适用场景: 需要深入观察子程序内部执行细节时使用。

缺点: 如果子程序很长, 会非常繁琐。

(2) P 指令 (Proceed)

作用: 单步执行指令, 但不进入子程序。

特点: 每按一次 P, 执行一条指令。当遇到 CALL 调用子程序时, 会一次性执行整个子程序, 然后停在 CALL 的下一条指令上。

特别之处：这是最常用的单步调试命令，能避免陷入子程序内部。

适用场景：调试主程序流程时，希望跳过已知正确的子程序，快速观察整体逻辑。

(3) G 指令 (Go)

作用：连续执行程序，直到遇到断点或程序结束。

特点：可以指定执行地址，例如 G 0100 表示从 CS:0100 开始连续运行。也可以设置断点，例如 G 0123 表示执行到地址 0123 处停止。

特别之处：是“运行”而不是“单步”，适合快速让程序跑到指定位置。

适用场景：想让程序快速执行到某个关键位置，或者直接运行完整程序时使用。

3. DOS 中断调用 (INT 21H) 的常用功能

AH=9: 显示以 '\$' 结尾的字符串

AH=10: 带缓冲的键盘字符串输入

AH=2: 显示单个字符

AH=4CH: 程序正常退出

4. 子程序的设计与调用

在 8086 汇编语言编程中，子程序 (Procedure) 是实现模块化设计的重要手段。本实验中主要采用**近过程 (NEAR)**的方式定义子程序，使用 PROC...ENDP 伪指令对子程序进行封装。

定义近过程子程序的基本格式为：

子程序名 PROC NEAR

..... (子程序主体代码)

RET

子程序名 ENDP

通过 CALL 子程序名 指令可以实现对子程序的调用。执行 CALL 指令时，CPU 会自动将返回地址 (当前 IP 值) 压入栈中，子程序执行完毕后，通过 RET 指令从栈中弹出返回地址，从而实现程序的正确返回。

由于子程序和主程序会共用 AX、BX、CX、DX、SI、DI 等寄存器，为了防止子程序修改这些寄存器的值而破坏主程序的运行环境，在进入子程序时必须使用 PUSH 指令将被使用的寄存器依次压入栈中进行保护 (称为**现场保护**)；在子程序执行结束、返回之前，再使用 POP 指令按照相反的顺序将寄存器值弹出恢复 (称为**现场恢复**)。这一操作是汇编语言子程序设计中的重要规范，必须严格遵守。

子程序与主程序之间的参数传递和结果返回，通常有以下两种常用方式：

- **通过标志位传递：**例如使用进位标志 CF。通过 STC 指令置位进位标志 (表示

某种状态，如“是闰年”），通过 CLC 指令清零进位标志（表示“不是闰年”），主程序再用 JC 或 JNC 指令进行判断。

- **通过寄存器传递：**子程序将计算结果放入 AX、BX 等通用寄存器中，返回后主程序直接从这些寄存器中获取结果。